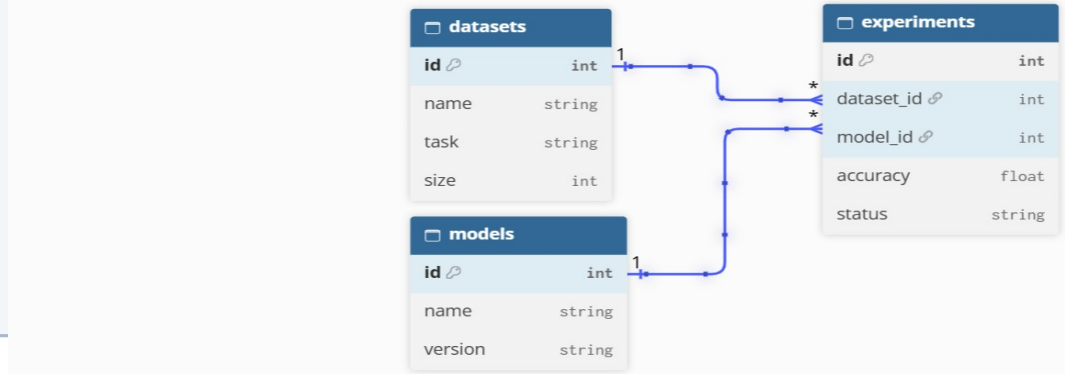
MiniDB

FILE-BASED DATABASE ENGINE / LAB REPORT №1

01 / DATABASE SCHEMA

MiniDB uses three related tables: datasets, models and experiments. Each experiment references one dataset and one model through foreign keys.



02 / STORAGE FORMAT

FILE	PURPOSE
schema.txt	Table definitions, columns, primary keys (PK) and foreign keys (FK).
meta.txt	Metadata: next auto-increment ID for each table.
<table>.dat	Persistent row storage; fields are tab-separated, with backslash escaping.
<table>.idx	Binary primary-key index: ID → byte offset in the corresponding .dat file.

MiniDB / IMPLEMENTATION & TESTS

CRUD OPERATIONS • INDEX BENCHMARK • BACKUP / RESTORE

03 / CRUD EXAMPLES

PYTHON API · EXAMPLE

```
db = MiniDB("data")
init_default_schema(db)

db.insert("datasets", {
    "name": "MNIST", "task": "classification", "size": 70000
})
db.select("experiments",
          where=Condition("status", "=", "completed"))
db.update("models", 1, {"version": "1.1"})
db.experiments_with_names()  # INNER JOIN
```

The CLI supports table listing, SELECT with filters, INSERT, UPDATE, DELETE and a JOIN view. Primary keys are unique; foreign keys are validated on write, and deleting a referenced parent row is blocked.

04 / INDEX BENCHMARK

Test setup: 1,000 rows · 200 ID lookups · 5 repetitions · time.perf_counter().

METHOD	MIN (s)	AVG (s)	MAX (s)
Sequential scan	0.3191	0.3272	0.3371
Indexed lookup	0.00510	0.00563	0.00629

≈ 58.3×

average speedup with the primary-key index

05 / BACKUP, RESTORE & LIMITATIONS

CLI · COMMANDS

```
python main.py backup data data_backup
python main.py restore data_backup data_restored
```

Backup is restored into a separate directory to avoid silently overwriting the active database. Current limitations: no SQL parser, transactions or concurrent multi-user access; SELECT/filtering scan the full table, while UPDATE/DELETE rewrite the table file. Benchmark values are machine-dependent and should be verified against the current benchmark.py output.